CSCI 580 - Artificial Intelligence
Final Report
Multilayer Perceptron on MNIST Handwritten Images

Joseph Lorenzo Bautista & Jesus Javier Ramirez
Kevin Xiong & Adam Lepage

May 15th, 2026

Introduction

Project Overview

This project analyzes and examines the use of *Multi Layer Perceptron (MLP)* neural network for handwritten digit recognition. In our evaluation, we trained out MLP on the MNIST dataset, which is a known and widely used benchmark of over $70,000$ grayscale images of handwritten digits, and evaluated its performance on both the standard MNIST test set and our custom group set of handwritten digit images collected by ourselves and peers.

The overall goal of the project can be categorized and split into these 2 main objectives:

1. To design and tune an MLP that is able to achieve and get strong accuracy on MNIST.
2. Assess and evaluate how well that model generalizes to real world hand written images that differs from the perfect, clean, and standardized MNIST data.

Reason and Motive

Recognition of handwritten digits is a foundational problem in fields like computer vision and machine learning. And although the MNIST dataset is considered to be the baseline and solved by current and modern deep learning standards, training on MNIST is still a great way towards studying how neural networks learn, how hyperparameters affect performance, and how models act and behave when tested on real world data outside.

So by collecting our own datasets of handwritten digit images and executing them through the trained model, we were able to directly compare benchmark performance against real world performance.

Approach

Our solution was developed and constructed in `PyTorch`, using only fully connected (dense) neural network layers. No convolutional layers (CNNs) or any other models/architectures were used.

We constructed and followed an iterative development process of:

1. **Iteration #1 - Baseline:** a simple 2 hidden layer MLP that's trained on MNIST to set up a reference accuracy and foundation.
2. **Iteration #2 - Larger Network + Dropout:** expanded the hidden layers and added dropout regularization in order to improve generalization.
3. **Hyperparameter Sweep:** executed and performed a systematic search across hidden layer sizes, dropout values, and learning rates in order to identify the best configuration.
4. **Iteration #3 - Tuned Architecture + Data Augmentation:** utilized the best hyperparameters from the sweep, integrated with random rotations and translations used to the training data towards improving stability and durability on the real world images.

And after each iteration, we calculated and measured validation accuracy during training, evaluated the model on the MNIST test set, and tested on the all of the group's handwritten images. We also generated a confusion matrix to identify which digits the model most often confused, and calculated a performance gap analysis, comparing per digit accuracy on MNIST vs the group's images. Additionally, misclassified examples and per digit accuracy results were used to steer each successive iteration.

High Level Framework of Solution

Here, we'll describe the overall structure of our solution and the final set of hyperparameters used in our best performing model.

Pipeline Overview

Our solution consists of four main stages, which are:

1. **Data Loading and Preprocessing:** loading the MNIST dataset using PyTorch's "library" of `torchvision.datasets.MNIST`, and load the group's custom handwritten images using a helper function we implemented, `ProjectDataLoader`. All of the images are normalized and resized to 28×28 grayscale in order to match the MNIST dataset format.
2. **Model Construction:** constructed a *Multi-Layer Perceptron* using Pytorch's `nn.Module`. And the MLP consists only of fully connected (linear) layers with non linear activations and dropout regularization.
3. **Training and Validation:** fully trained the model on $50,000$ MNIST images while validation used $10,000$ images after every epoch. Loss is calculated using Cross Entropy Loss and weights are updated using the Adam optimizer.
4. **Evaluation:** evaluated the trained model on:
   a) The standard $10,000$ image MNIST test set.
   b) The group's collected handwritten digit images.

   Accuracy results and statistics are calculated both overall and per digit. A confusion matrix is generated in order to identify which digits are most often confused with one another, and a performance gap analysis measures the difference between MNIST and the group image accuracy on a per digit basis in order to evaluate real world generalization.

Final Neural Network Architecture

After 3 iterations of development and a systematic hyperparameter sweep, the final architecture and structure is a fully connected feedforward network with 2 hidden layers. As a table, we can see how it look like in the following figure below.

We can visualize it as a table:

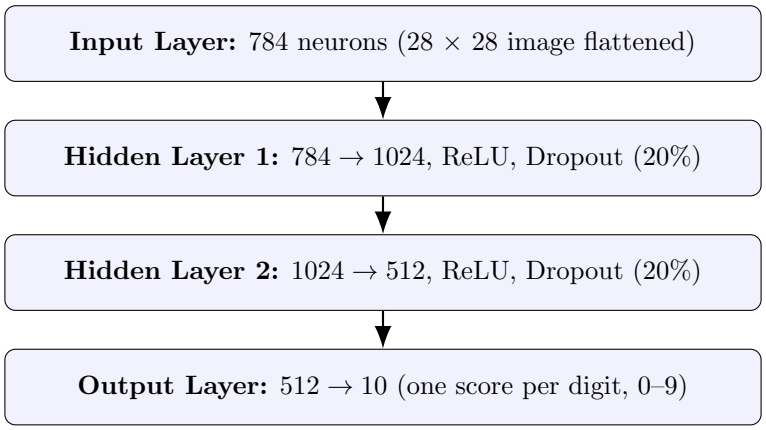


Figure 1: Final MLP Architecture.

Counting all the weights and biases in every layer of our model's network. Each `nn.Linear` layer has 2 sets of parameters:

a) a weight matrix

b) a bias vector

And for each layer, the parameter count is calculated by:

$$parameter = (input\ size \times output\ size) + output\ size$$

The first part is the weights which is one weight per input-out connection, and the second part being the biases which is one bias per output neuron.

Layer	Computation	Parameters
`fc1` (784 → 1024)	(784 × 1024) + 1024	803,840
`fc2` (1024 → 512)	(1024 × 512) + 512	524,800
`output` (512 → 10)	(512 × 10) + 10	5,130
Total		**1,333,770**

Table 1: Trainable parameters per layer in the final MLP architecture.

The model contains approximately 1.3 million trainable parameters (weights and biases). The output layer makes raw logits which is the unnormalized scores, which are evaluated as class predictions by picking the highest scoring output. Additionally, we purposely left out a final softmax activation in the model since PyTorch's `CrossEntropyLoss` applies log softmax internally.

Final Hyperparameters

The following table below is a summary of our set of hyperparameters used in our model.

Hyperparameter	**Value**	**Notes**
Hidden Layer Sizes	$[1024, 512]$	From hyperparameter sweep
Activation Function	ReLU	After each hidden layer
Dropout Rate	0.2	Between hidden layers
Optimizer	Adam	Default $\beta = (0.9, 0.999)$
Learning Rate	0.001	Confirmed via sweep
Loss Function	Cross-Entropy	Multi-class classification
Batch Size	256	GPU memory permitting
Epochs	15	Convergence with augmentation
Data Augmentation	Rotation $\pm 10°$, shift $\pm 10\%$	Training set only
Train / Validation Split	50,000 / 10,000	From 60k training images
Normalization	mean = 0.5, std = 0.5	Pixel values $\rightarrow [-1, 1]$

Table 2: Final hyperparameters used in the best-performing model.

Tools and Libraries

The entire program and solution is implemented in `Python` using the following libraries:

1. `PyTorch:` neural network construction, training, and GPU acceleration.
2. `torchVision:` MNIST dataset loading and image transformations.
3. `NumPy:` array operations.
4. `Pandas:` tabular data manipulation for result analysis
5. `scikit-learn:` confusion matrix calculation
6. `PIL (Pillow):` image file reading for the group's PNG and JPG files.
7. `Matplotlib and Seaborn:` visualization and result plots/charts.

The `.ipynb` was developed and ran on Google Colab. And while the program was made and structured for GPU usage when available, the results we produced and obtained, were on CPU.

From Baseline to Final Model

Rather than trying to find the most optimal model in a single attempt, we chose an iterative approach. With each iteration introducing one or more changes, architecture size, regularization, hyperparemeter tuning, data augmentation, and the effect of each change was measured against the previous and past iteration.

This allowed us to attribute improvements to specific design choices and produced a clear progression of results, which is shown and analyzed in the rest of the report.

Detailed Aspect of Solution

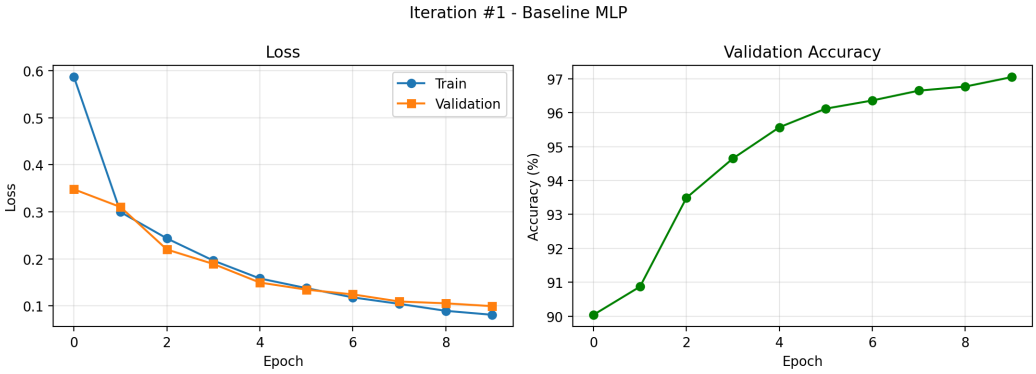
Overview

The model was trained across three different iterations. Each improving from the previous iterations. All training used the MNIST set with a total of 60k images. Within that 60k images, 50k was used for training and 10k was for testing. The training loop follows a standard training cycle with a forward pass which makes the 10 output scores (one digit per class), CrossEntropyLoss measures the error, backpropagation calculates the weight, and Adam optimizer updates weights to reduce loss. This repeats every epoch.

Iteration #1: Baseline MLP

The first iteration was a baseline MLP with no regularization. The model used 784 input neurons (flattened 28×28 images) and two hidden layers with one being 128 and the other being 64 neurons. It also used ReLU activation and had 10 output layers. It was trained for 10 epochs with a learning rate of 0.001.

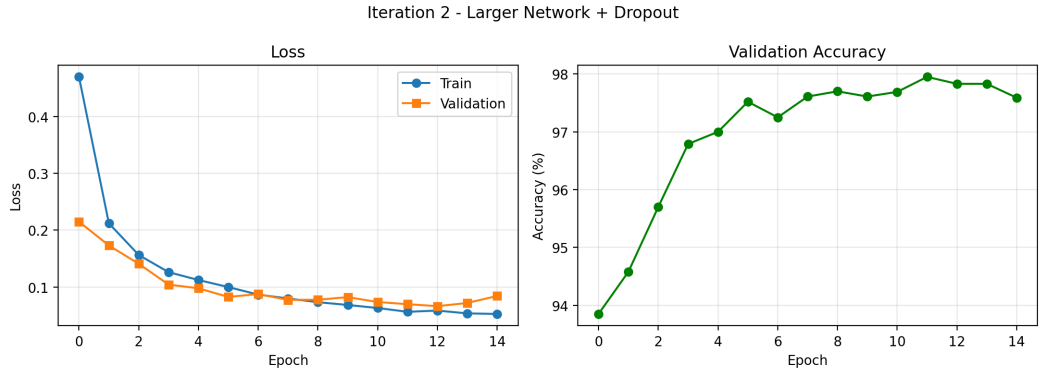
Both the training and validation loss decreased at a constant rate, and the model got to a final MNIST test accuracy of about 96.80%.



Iteration #2: Larger Network + Dropout

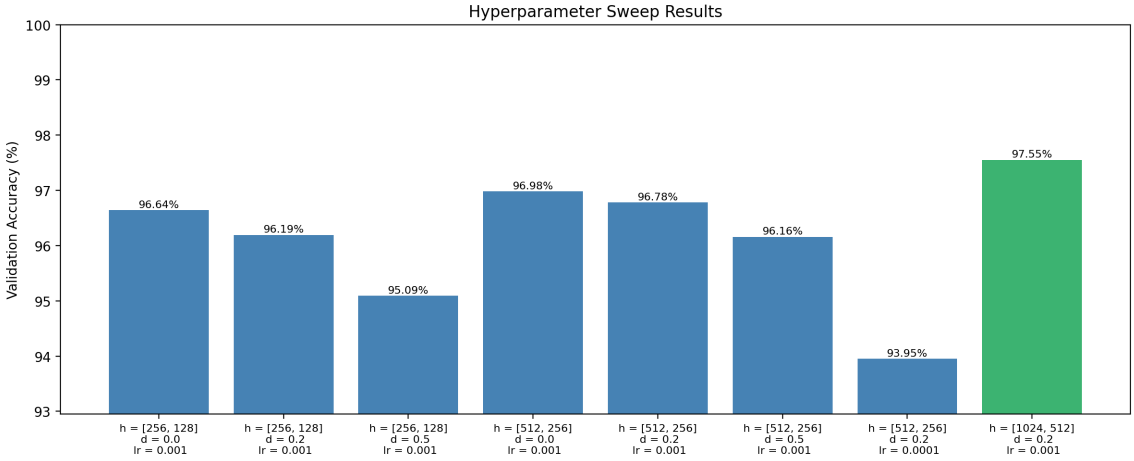
With the second iteration, we increased the number of neurons in the two hidden layers. Instead of the 128 and 64 neurons we had previously in iteration 1, we now have 512 and 256 neurons. This second iteration also includes a 20% dropout rate to help with the overfitting. Training was increased from 10 to 15 epochs.

The larger network gave the model more power and the added dropout rate prevents the model from relying too much on certain neurons. The training and validation loss were really close to each other throughout the training which shows that the model was learning with little to no overfitting. With the changes, we saw an improvement to 97.3%.



Hyperparameter Sweep

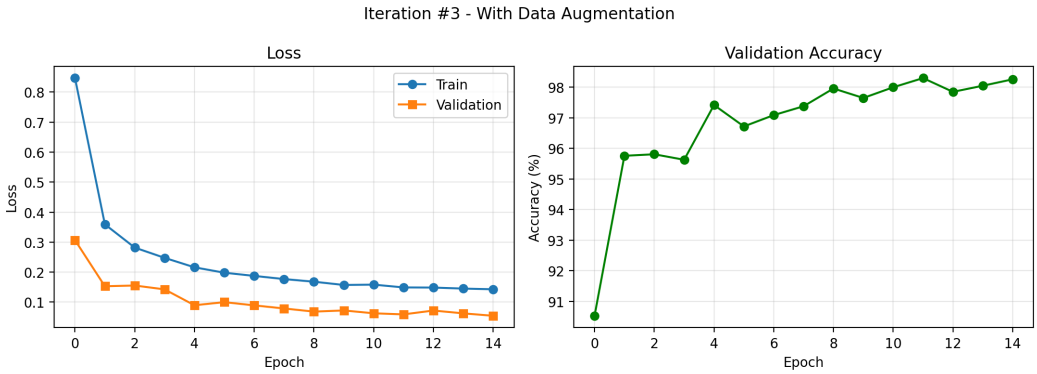
Before we head onto iteration 3, a hyperparameter sweep was done to test different numbers of neurons, hidden layer sizes, dropout rates, and learning rates. Eight different combinations was tested and was ranked on validation accuracy. The best combination used two hidden layers with 512 and 256 neurons, a dropout rate of 20% and a learning rate of 0.001. This is the same setup in iteration 2.



Iteration #3: Data Augmentation

The third iteration and our best performing one used the setup from the hyperparameter sweep and added data augmentation to it.The data augmentation randomly rotates the images 10 degrees and moves images around 10% in each direction.This was done to make sure the model actually learns and not just memorizes. The augmented images were harder to learn from initially so the

training loss at the start was a lot higher than the previous iterations. But the model eventually converged to a better final accuracy of 98.2%



Key Observations From Results:

- The loss steadily decreased in all three iterations. Shows that the model continues to learn during the training.
- Validation loss stays close to the training loss in each iteration. Means that the model was not overfitting a lot.
- Iteration 3 started with higher training loss due to data augmentation which made it harder to learn initially. However final accuracy was higher than the other two iterations.
- MNIST accuracy improved from every iteration onwards. From 96.8% $\rightarrow$ 97.3% $\rightarrow$ 98.2%.

Test Results and Analysis

Comparison and Analysis Across Iterations

The performance of the model improved consistently across the three development iterations, as shown in the training/validation curves and accuracy plots.

- Iteration #1 – Baseline MLP
  a) The initial model established a strong baseline, achieving 96.8% validation accuracy. However, this model had limited generalization capability due to its relatively simple architecture and lack of regularization.
- Iteration #2 – Larger Network + Dropout
  a) Expanding the hidden layer sizes and introducing dropout (20%) improved performance to 97.31% validation accuracy. Dropout reduced overfitting by preventing the network from relying too heavily on specific neurons. Validation loss closely tracked training loss, indicating improved generalization compared to the baseline.

- Iteration #3 – Tuned Model + Data Augmentation

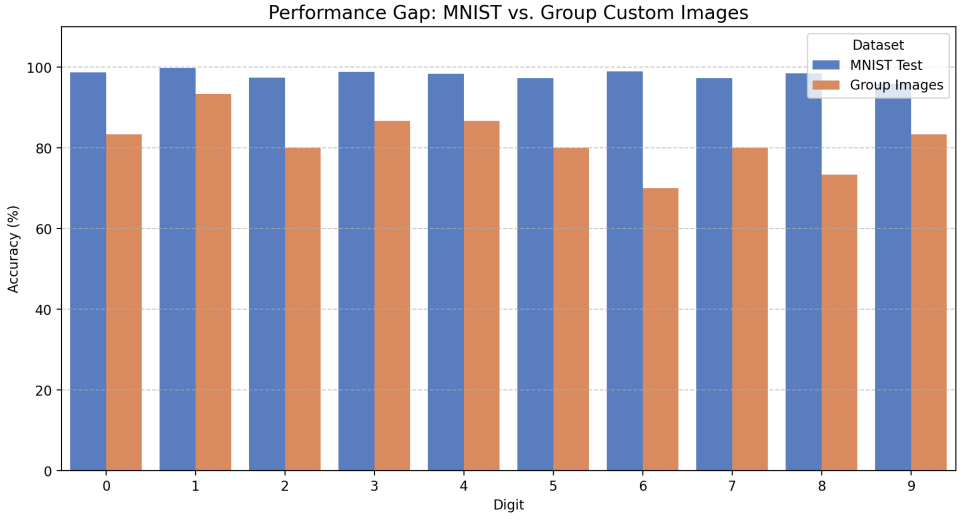
    a) The final iteration incorporated data augmentation, did random rotations and shifts along with optimized hyperparameters. This resulted in the best performance of 98.21% validation accuracy. The steady decrease in loss across all iterations and the improved validation accuracy confirm that each modification contributed positively to learning. Data augmentation was particularly important in helping the model learn more robust and invariant features rather than memorizing pixel patterns.

The Overall Trend for the MNIST Accuracy Across Iterations was:

- Iteration #1: 96.80%
- Iteration #2: 97.31%
- Iteration #3: 98.21%

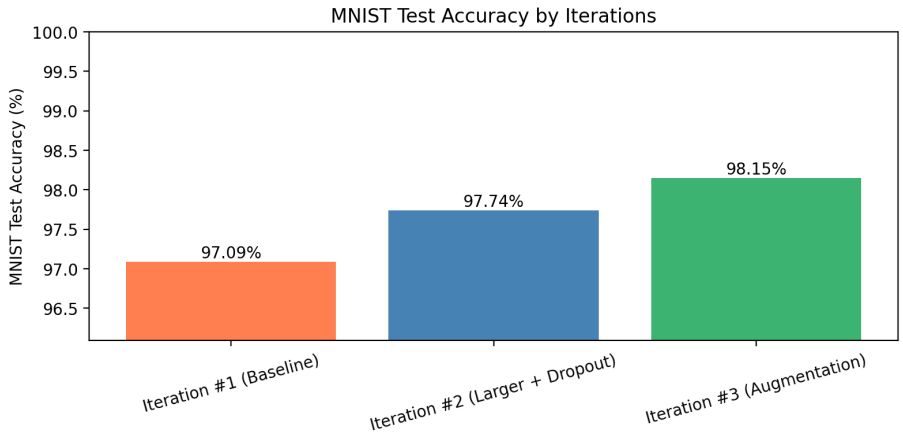
These percentages demonstrate that increasing the model capacity improves learning capability, adding dropout improves generalization and data augmentation is critical for robustness to real world variations.

Below, compares per-digit accuracy on MNIST versus our group's images. MNIST accuracy stays around **97-100%** across all digits, while group accuracy ranges from **70% to 93%**. The smallest gap is for digit **1** (∼7%) and the largest for digit **6** (∼29%). This drop reflects **distribution shift**, the model generalizes less effectively to handwriting that differs from MNIST's clean, centered, stroke-normalized style.



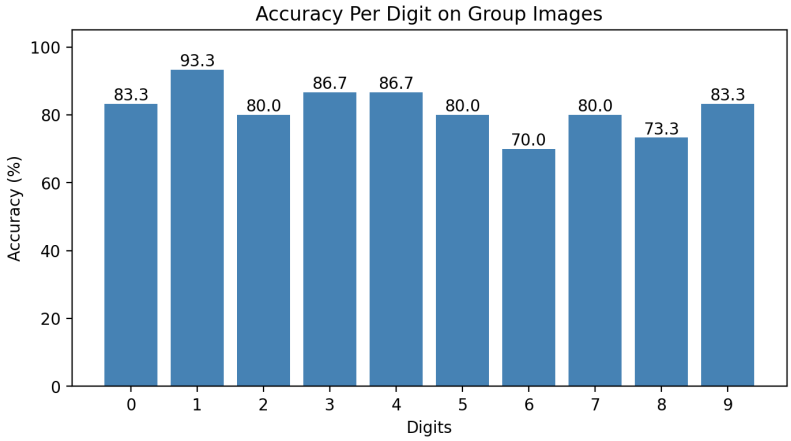
Detailed Analysis of Test Results on Group Data

While the model performed very well on MNIST, its performance dropped when evaluated on real-world handwritten images. MNIST accuracy is consistently near 98–100% across all digits, group dataset accuracy is significantly lower, averaging ∼ 81.67%. This shows a substantial generalization gap between standardized and real-world data.



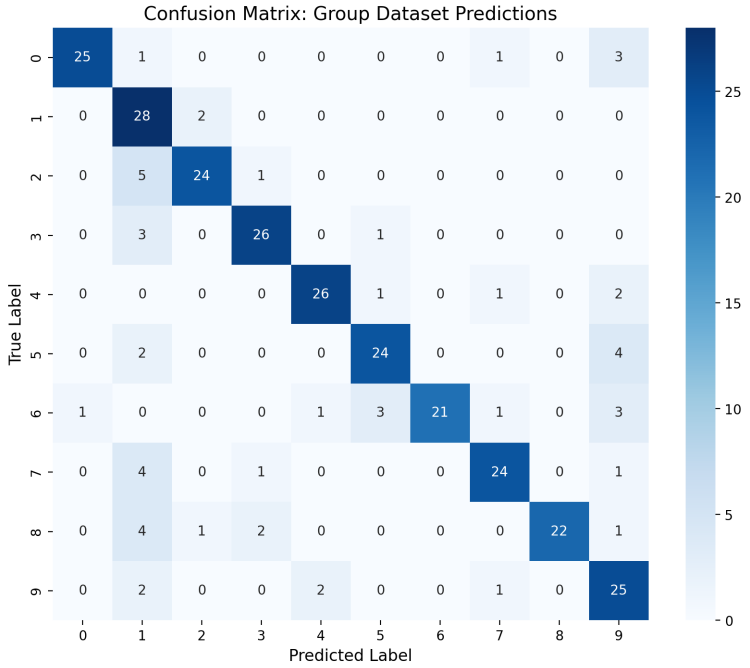
Per-Digit Accuracy Analysis:

Some digits achieved relatively high accuracy, indicating that the learned features generalized well for simpler patterns.



- High-Performing Digits:
  - Digit 1 ($\sim$ 93.3%)
  - Digits 3, 4 ($\sim$ 86.7%), 0, 9 ($\sim$83.3%)
- Lower-Performing Digits:
  - Digit 6 ($\sim$ 70.0%)
  - Digit 8 ($\sim$ 73.3%)
  - Digits 2, 5, 7 ($\sim$ 80%)

The confusion matrix below shows that the final model classified each digit in our group's dataset. Diagonal entries are correct predictions, while off diagonal entries are the misclassifications. Below, we can see that, Digit 1 had the highest accuracy (28/30) and digit 6 the lowest (21/30).



The most common errors were 2s misclassified as 1s (5 cases), 5s as 9s (4 cases), and 0s as 9s (3 cases), pattern that reflect shared visual features such as vertical strokes and closed loops. Overall, the errors are not random but cluster around digits with similar shapes, consistent with the limitation of an MLP that can't exploit local spatial structure.

This variation indicates that the model struggles more with digits that:

- Have similar shapes
- Contain loops or complex strokes
- Vary significantly in handwriting style
- Misclassification Analysis

These errors show that the model is highly sensitive to:

- Stroke thickness
- Incomplete or distorted shapes
- Writing style variations not present in MNIST

What Worked Well:

- The model learned strong general digit representations from MNIST.
- Data augmentation improved robustness to minor transformations.
- Digits with simple and consistent shapes were classified with high accuracy.
- Training and validation curves across all iterations show stable convergence with minimal overfitting.

What Did Not Work Well:

- Significant performance drop when transitioning to real-world data.
- Poor performance on complex or ambiguous digits like 6 or 8,
- Sensitivity to handwriting variability, including:
  - Misalignment
  - Noise
  - Non-standard shapes

Conclusion and Future Work

Conclusion

This project showed that a fully connected multilayer perceptron can learn handwritten digit recognition effectively on MNIST with the right techniques. Starting from the baseline MLP performance improved from 96.8% validation accuracy to 97.31% after adding a larger network and 20% dropout, and then finally to 98.21% after tuning hyperparameters and applying data augmentation with random rotations and shifts. The final model used a $784 \rightarrow 1024 \rightarrow 512 \rightarrow 10$ architecture and about 1.33 million trainable parameters.

Moving on to the handwritten test images reveals a generalization gap. While the model performed at over 98% on MNIST data, the accuracy dropped to about 81.67 on the handwritten dataset. This shows that the network was sensitive to the handwritten variation, possibly due to stroke thickness, incomplete digits, and digits with more complex forms, such as 6 and 8.

Overall, the iterative approach was effective because each change had a measurable impact on the model's accuracy. Increasing the model capacity improved learning, dropout reduced overfitting, and augmentation improved robustness. However, the results also showed that a dense MLP by itself is not fully sufficient for reliable handwritten digit recognition.

Future Work

A natural next step would be comparing this MLP against a convolutional neural network since they are usually better at learning spatial patterns in images and may perform better on handwritten digits. In addition, collecting a larger and more diverse handwritten dataset would likely improve generalization by exposing the model to more writing styles, thicknesses, and noisy samples.

Another improvement would be greater preprocessing and augmentation. For example, centering, thresholding, and adding more varied augmentations such as scaling or noise could make the model more resistant to distortions that don't appear in the more uniform MNIST dataset. Future work also could include a more detailed error analysis by digit, along with additional metrics like precision or recall, which may help understand which digits remain hardest to classify